LAPORAN JOBSHEET
“14 - Sistem Autentikasi & Proteksi Route”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Langkah Praktikum

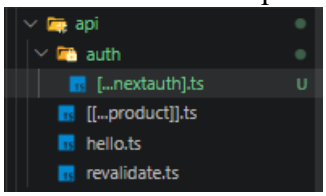
Bagian 1 – Install NextAuth

1. npm install next-auth –force

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm install next-auth --force
npm warn using --force Recommended protections disabled.

up to date, audited 488 packages in 4s

158 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

Bagian 2 – Konfigurasi API Auth

1. Buat file dan folder pada folder pages/api/auth/[...nextauth].ts



2. Modifikasi file […nextauth].ts:

P2 > Praktikum > next-routing > src > pages > api > auth > [...nextauth].ts > default

```
import NextAuth, {NextAuthOptions} from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions: NextAuthOptions = {
    session: {
        strategy: "jwt",
    },
    secret: process.env.NEXTAUTH_SECRET,
    providers: [
        CredentialsProvider({
            name: "credentials",
            credentials: {
                fullname: { label: "Full Name", type: "text" },
                email: { label: "Email", type: "text" },
                password: { label: "Password", type: "password" },
            },

            async authorize(credentials) {
                /* eslint-disable @typescript-eslint/no-explicit-any */
                const user : any = {
                    id: 1,
                    name: credentials?.fullname,
                    password: credentials?.password,
                    email: credentials?.email,
                }
                if (user) {
                    return user;
                } else {
                    return null;
                }
            },
        }),
    ],

    callbacks: {
        async jwt({ token, account, profile, user }) {
            if (account?.provider === "credentials" && user) {
                token.email = user.email;
            }
            return token;
        },
        async session({ session, token } : any) {
            if (token.email) {
                session.user.email = token.email;
            }
            return session;
        }
    },
};

export default NextAuth(authOptions);
```

Bagian 3 – Tambahkan Secret

1. Buka file .env.local dan tambahkan code pada line 12
   - NEXTAUTH_SECRET=RANDOM_BASE64_STRING
   - Untuk mendapatkan nilai RANDOM_BASE64_STRING gunakan generator RANDOM_BASE64_STRING seperti https://www.convertsimple.com/randombase64-generator/

```
P2 > Praktikum > next-routing > .env.local
1   FIREBASE_API_KEY=AIzaSyDen7Mk5dq0CLAuXfcwnlvBLDZrmualWTQ
2   FIREBASE_AUTH_DOMAIN=framework-nextjs.firebaseapp.com
3   FIREBASE_PROJECT_ID=framework-nextjs
4   FIREBASE_STORAGE_BUCKET=framework-nextjs.firebasestorage.app
5   FIREBASE_MESSAGING_SENDER_ID=348929355692
6   FIREBASE_APP_ID=1:348929355692:web:ae9edc3db1add71bad5409
7
8   REVALIDATE_TOKEN=12345678
9
10  // saya menggunakan https://generate-secret.vercel.app/32 untuk generate secret key
11  NEXTAUTH_SECRET=7fa8447cb7e49fbfcf827b7224fc1c22
```

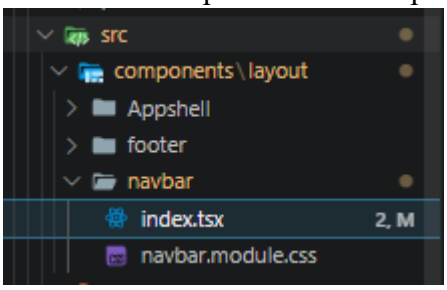
Bagian 4 – Tambahkan SessionProvider

1. Buka file _app.tsx dan modifikasi:

```
P2 > Praktikum > next-routing > src > pages > _app.tsx > ...
import "@/styles/globals.css";
import type { AppProps } from "next/app";
import Appshell from "@/components/layout/Appshell";
import { SessionProvider } from "next-auth/react";

export default function App({ Component, pageProps: { session, ...pageProps } }:
  return (
    <SessionProvider session={pageProps.session}>
      <Appshell>
        <Component {...pageProps} />
      </Appshell>
    </SessionProvider>
  );
}
```

Bagian 5 – Tambahkan Tombol Login & Logout

1. Buka index.tsx pada folder component/navbar:



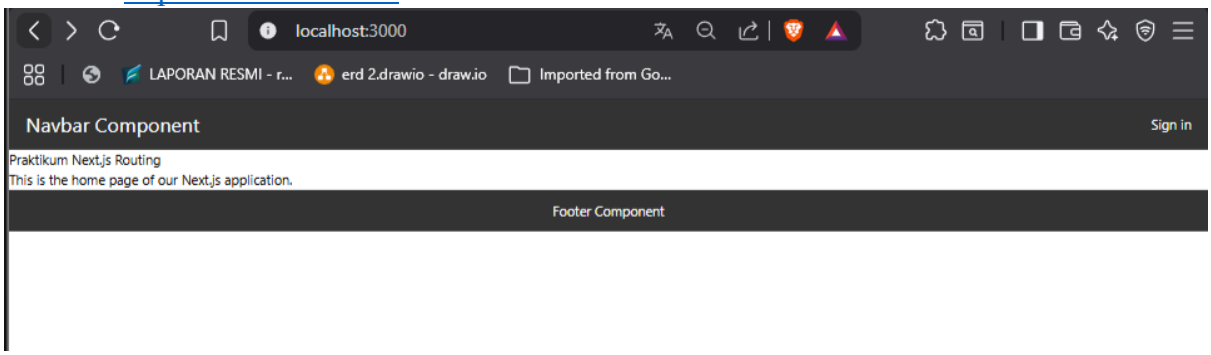
2. Modifikasi file index.tsx pada line 10 dan 2

```
P2 > Praktikum > next-routing > src > components > layout > navbar > index.tsx > ...
1   import styles from "./navbar.module.css";
2   import { signIn, signOut, useSession } from "next-auth/react";
3
4   const Navbar = () => {
5     return (
6       <div className={styles.navbar}>
7         <div className="big">
8           <h1>Navbar Component</h1>
9         </div>
10        <div>
11          <button onClick={() => signIn()}> Sign in</button>
12        </div>
13      </div>
14    );
15  };
16
17  export default Navbar;
```

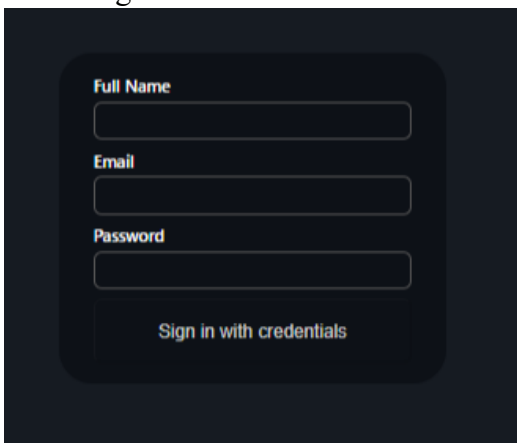
3. Buka file file navbar.module.scss tambahkan code pada line 9

```
> next-routing > src > components > layout > navbar > navbar.module.cs
1   .navbar {
2       width: 100%;
3       height: 60px;
4       background-color: #333;
5       color: white;
6       display: flex;
7       justify-content: center;
8       align-items: center;
9       padding: 0 20px;
10      justify-content: space-between;
11  }
```

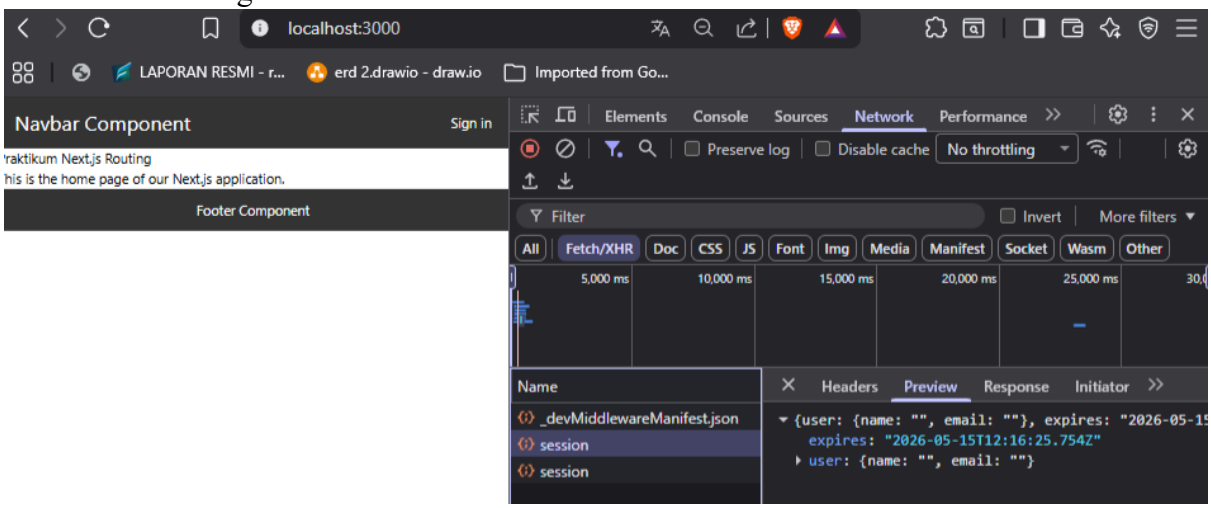
4. Jalankan http://localhost:3000/



5. Jika di klik sign in maka akan muncul dan isikan textbox masing. Setelah itu klik button sign in dan setelah diklik maka akan kembali ke halaman localhost

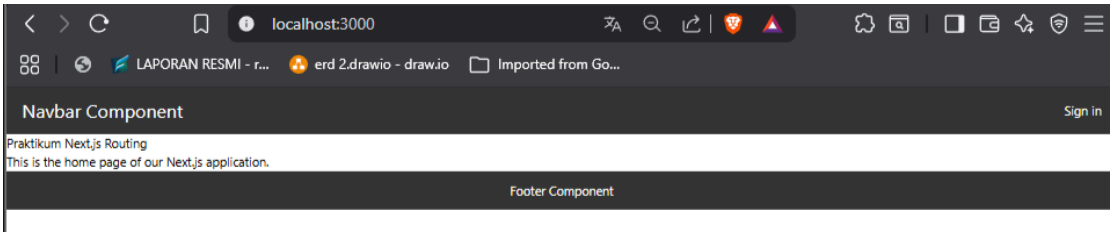


6. Setelah berhasil sign in maka akan muncul session

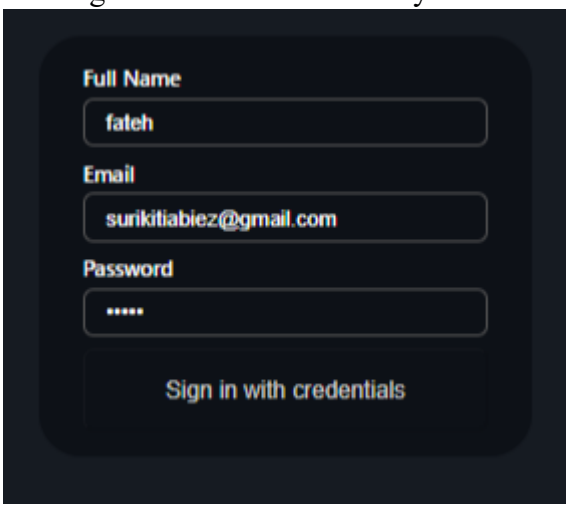


7. Untuk dapat menangkap data pada session maka tambahkan code sebagai berikut

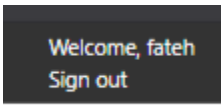
8. Uji coba sign in dan sign out
   - Jalankan Kembali npm run dev
   - Jalankan localhost



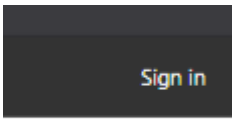
- Klik sign in dan isikan textboxnya



- Maka akan muncul tombol signout



- Ketika user klik signout maka akan kembali sign in



Menambahkan Data Tambahan (Full Name)

1. Buka file [...nextauth].js dan tambahkan code pada line 22

```
22 async authorize(credentials) {
       /* eslint-disable @typescript-eslint/no-explicit-any */
       const user : any = {
           id: 1,
           name: credentials?.fullname,
           password: credentials?.password,
           email: credentials?.email,
       }
       if (user) {
           return user;
       } else {
```

2. Pada callbacks modifikasi codenya menjadi berikut :

```
callbacks: {
    async jwt({ token, account, profile, user }) {
        if (account?.provider === "credentials" && user) {
            token.email = user.email;
            token.fullname = user.fullname;
        }
        return token;
    },
    async session({ session, token } : any) {
        if (token.email) {
            session.user.email = token.email;
        }
        if (token.fullname) {
            session.user.fullname = token.fullname;
        }
        return session;
    }
},
```

3. Modifikasi navbar.module.scss

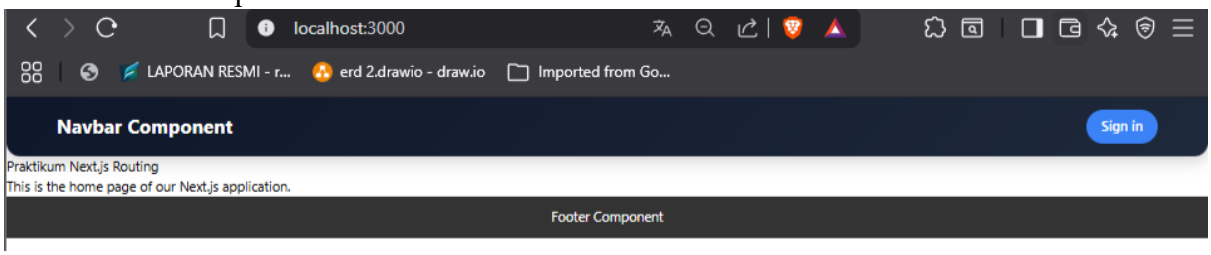
P2 > Praktikum > next-routing > src > components > layout > navbar > navbar.module.css > .navbar

```
.navbar {
  width: 100%;
  height: 70px;
  padding: 0 60px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: linear-gradient(135deg, #0f172a, #1e293b);
  color: #ffffff;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
  border-bottom-left-radius: 16px;
  border-bottom-right-radius: 16px;
}

/* Logo / Brand */
.navbar__brand {
  font-size: 22px;
  font-weight: 700;
  letter-spacing: 0.5px;
}

/* Right Section */
.navbar__right {
  display: flex;
  align-items: center;
  gap: 32px;
}

/* Welcome text */
.navbar__user {
  font-size: 16px;
  font-weight: 500;
  opacity: 0.9;
}

/* Button */
.navbar__button {
  padding: 8px 18px;
  border-radius: 900px;
  border: none;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.3s ease;
}

/* Sign In */
.navbar__button--primary {
  background-color: #3b82f6;
  color: white;
}

.navbar__button--primary:hover {
  background-color: #2563eb;
  transform: translateY(-2px);
}

/* Sign Out */
.navbar__button--danger {
  background-color: #ef4444;
```

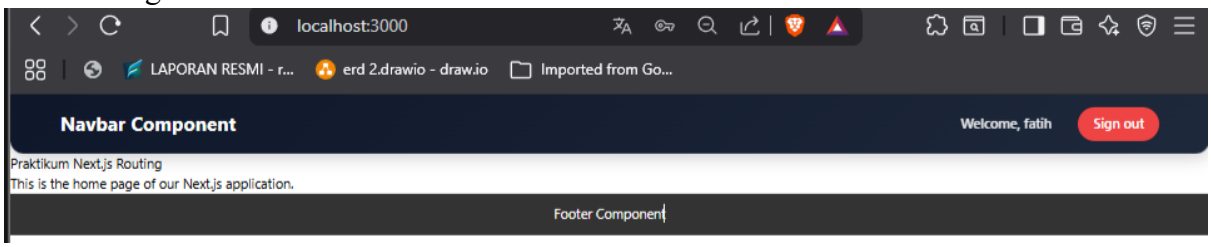
4. Modifikasi index.tsx pada folder components/layouts/navbar

```
const Navbar = () => {
  const { data } = useSession();
  return (
    <div className={styles.navbar}>
      <div className={styles.navbar__brand}>
        <h1 style={{ margin: 0 }}>Navbar Component</h1>
      </div>
      <div className={styles.navbar__right}>
        {data ? (
          <>
            <p className={styles.navbar__user}>Welcome, {data.user?.name}</p>
            <button
              className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
              onClick={() => signOut()}
            >
              Sign out
            </button>
          </>
        ) : (
          <button
            className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
            onClick={() => signIn()}
          >
            Sign in
          </button>
        )}
      </div>
    </div>
  );
};
```

5. Jalankan browser pada localhost



6. Lakukan sign in



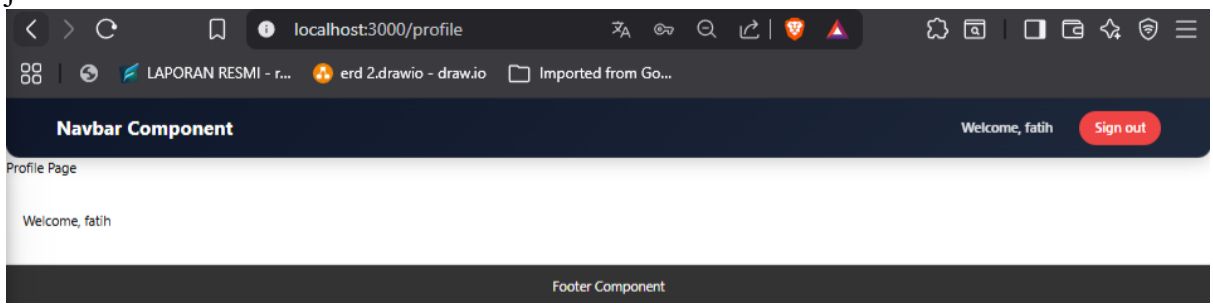
Proteksi Halaman Profile

Buat Halaman Profile

1. pages/profile/index.tsx dan modifikasi file index.tsx

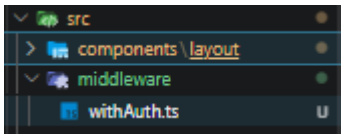
```
P2 > Praktikum > next-routing > src > pages > profile > index.tsx > default
import { useSession } from "next-auth/react";

const ProfilePage = () => {
    /* eslint-disable @typescript-eslint/no-explicit-any */
    const { data }: any = useSession();

    return (
        <div>
            <div className="tittle">
                <h1>Profile Page</h1>
                </div>
            <div className="container" style= {{ marginTop: '20px', margi
                <h1>Welcome, {data?.user?.name}</h1>
            </div>
        </div>
    );
};

export default ProfilePage;
```

2. jalankan browser



Buat Middleware Authorization

1. Buat file withAuth.ts dan folder dengan nama middleware di src



2. Modifikasi withAuth.ts

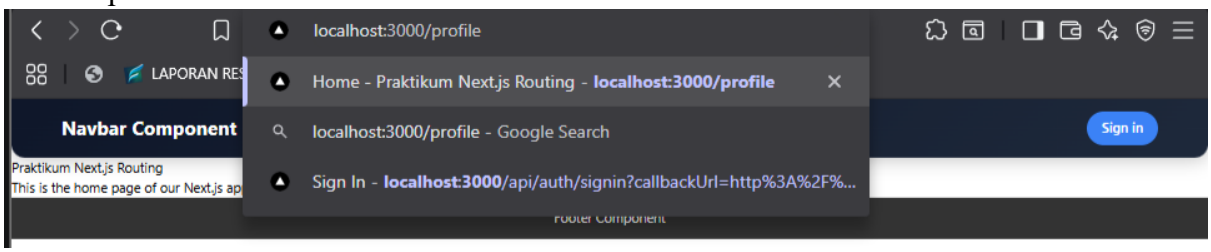
```
P2 > Praktikum > next-routing > src > middleware > withAuth.ts > withAuth > <function>
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware,NextRequest, NextResponse } from "next/server";

export default function withAuth(
    middleware: NextMiddleware,
    requireAuth: string[] = [],
) {
    return async (req: NextRequest, next: NextFetchEvent) => {
        const pathname = req.nextUrl.pathname;

        if (requireAuth.includes(pathname)) {
            const token = await getToken({
                req,
                secret: process.env.NEXTAUTH_SECRET,
            });

            if (!token) {
                const loginUrl = new URL("/auth/login", req.url);
                return NextResponse.redirect(loginUrl);
            }
        }
        return middleware(req, next);
    }
}
```

3. Modifikasi file middleware.ts

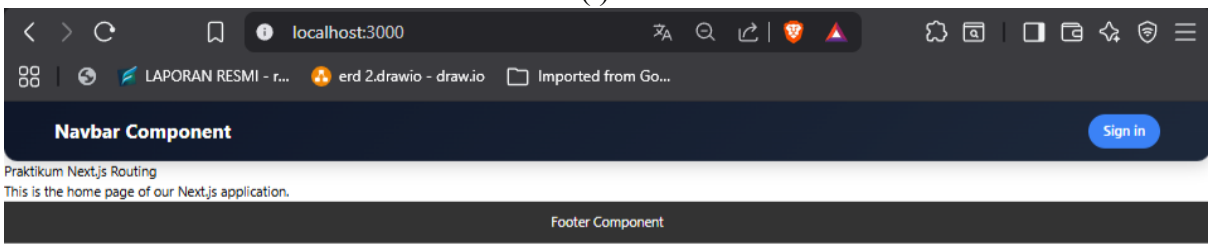
```ts
import { getToken } from "next-auth/jwt";
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Route yang memerlukan proteksi autentikasi
const protectedRoutes = ["/product", "/about"];
const profileRoutes = ["/profile"];

export async function middleware(request: NextRequest) {
  // Ambil JWT token dari NextAuth
  const token = await getToken({
    req: request,
    secret: process.env.NEXTAUTH_SECRET,
  });

  const { pathname } = request.nextUrl;

  // Cek apakah route saat ini adalah profile
  const isProfileRoute = profileRoutes.some((route) =>
    pathname.startsWith(route),
  );

  // Cek apakah route saat ini memerlukan proteksi
  const isProtectedRoute = protectedRoutes.some((route) =>
    pathname.startsWith(route),
  );

  // Jika belum login dan mengakses /profile, redirect ke / (home)
  if (!token && isProfileRoute) {
    return NextResponse.redirect(new URL("/", request.url));
  }

  // Jika belum login dan mengakses protected route lainnya, redirect ke /login
  if (!token && isProtectedRoute) {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  // Jika sudah login dan mengakses /auth/login, redirect ke /product
  if (token && pathname === "/auth/login") {
    return NextResponse.redirect(new URL("/product", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/product", "/about", "/auth/login", "/profile"],
};
```

Pengujian

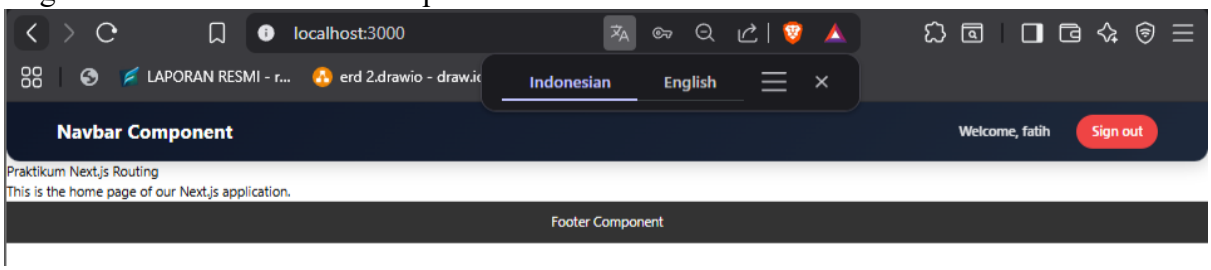
1. Uji 1 – Belum Login
   Akses: /profile Hasil: Redirect ke home



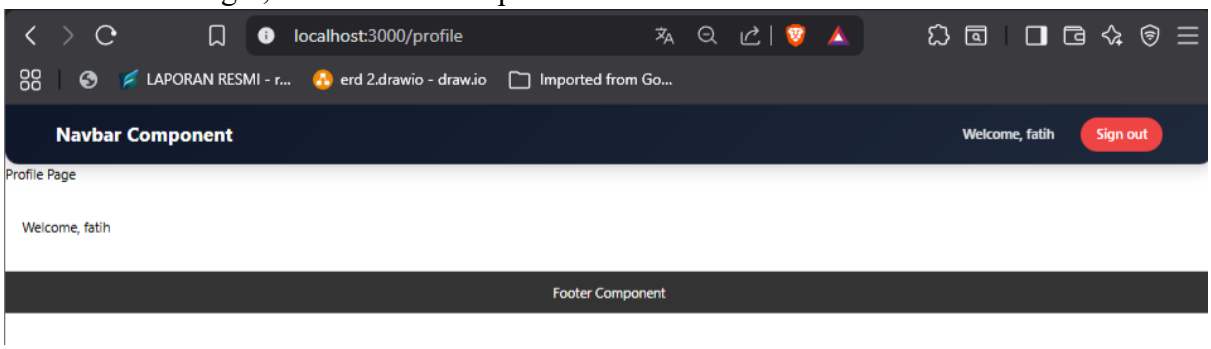
   Setelah di enter maka akan redirect ke home (/)



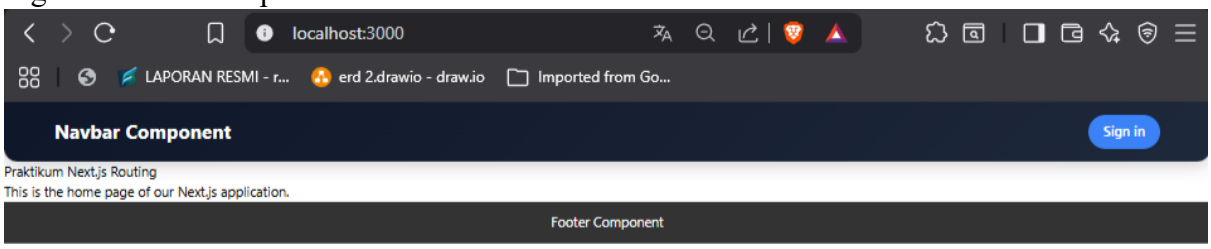
2. Uji 2 – Sudah Login
   Login terlebih dahulu → Akses /profile Hasil: Bisa masuk



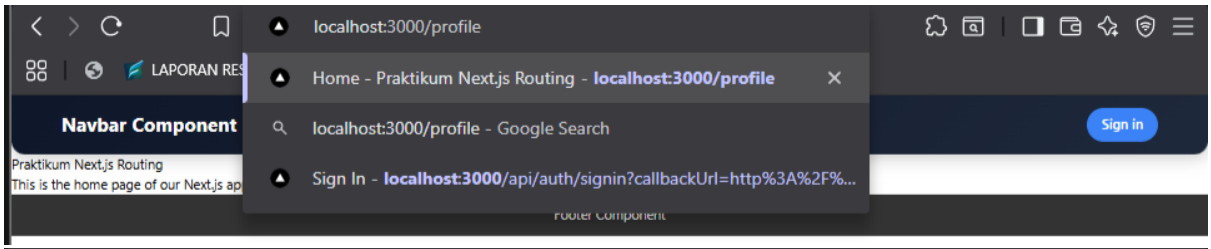
   Kondisi sudah login, kemudian akses /profile maka bisa masuk

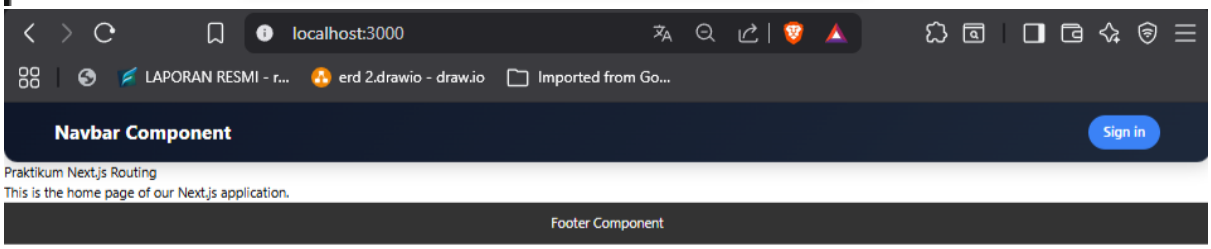


3. Uji 3 – Logout Klik
   Sign Out → Akses /profile Hasil: Tidak bisa masuk



   Kemudian akses /profile maka tidak akan bisa akses, karena cookies sudah terhapus





Tugas Praktikum

1. Implementasikan login menggunakan Credentials Provider

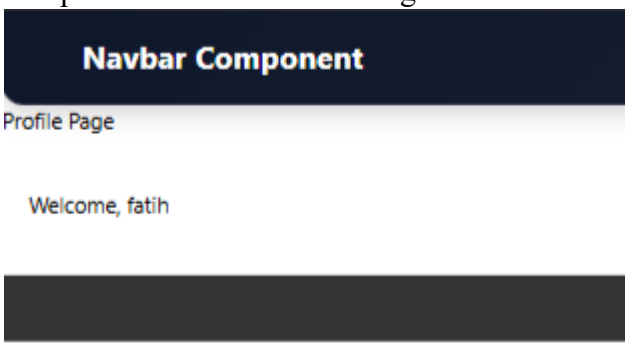
P2 > Praktikum > next-routing > src > pages > api > auth > [...nextauth].ts > ...

```python
import NextAuth, {NextAuthOptions} from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions: NextAuthOptions = {
    session: {
        strategy: "jwt",
    },
    secret: process.env.NEXTAUTH_SECRET,
    providers: [
        CredentialsProvider({
            name: "credentials",
            credentials: {
                name: { label: "Name", type: "text" },
                email: { label: "Email", type: "text" },
                password: { label: "Password", type: "password" },
            },

            async authorize(credentials) {
                /* eslint-disable @typescript-eslint/no-explicit-any */
                const user : any = {
                    id: 1,
                    name: credentials?.name,
                    password: credentials?.password,
                    email: credentials?.email,
                }
                if (user) {
                    return user;
                } else {
                    return null;
                }
            },
        }),
    ],

    callbacks: {
        async jwt({ token, account, profile, user }) {
            if (account?.provider === "credentials" && user) {
                token.email = user.email;
                token.name = user.name;
            }
            return token;
        },
        async session({ session, token } : any) {
            if (token.email) {
                session.user.email = token.email;
            }
            if (token.name) {
                session.user.name = token.name;
            }
            return session;
        }
    },
};

export default NextAuth(authOptions);
```

2. Tambahkan field full name.

```
callbacks: {
    async jwt({ token, account, profile, user }) {
        if (account?.provider === "credentials" && user) {
            token.email = user.email;
            token.name = user.name;
        }
        return token;
    },
    async session({ session, token } : any) {
        if (token.email) {
            session.user.email = token.email;
        }
        if (token.name) {
            session.user.name = token.name;
        }
        return session;
    }
},
```

3. Tampilkan full name setelah login.



4. Buat halaman profile.

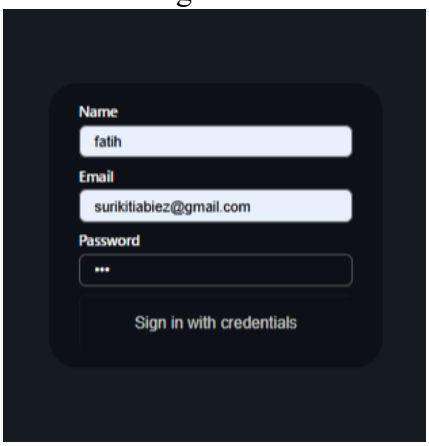
```
P2 > Praktikum > next-routing > src > pages > profile > index.tsx > ProfilePage
1   import { useSession } from "next-auth/react";
2
3   const ProfilePage = () => {
4       /* eslint-disable @typescript-eslint/no-explicit-any */
5       const { data }: any = useSession();
6
7       return (
8           <div>
9               <div className="tittle">
10                  <h1>Profile Page</h1>
11              </div>
12              <div className="container" style= {{ marginTop: '20px', margin
13                  <h1>Welcome, {data?.user?.name}</h1>
14              </div>
15          </div>
16      );
17  };
18
19  export default ProfilePage;
```

5. Lindungi halaman profile dengan middleware.

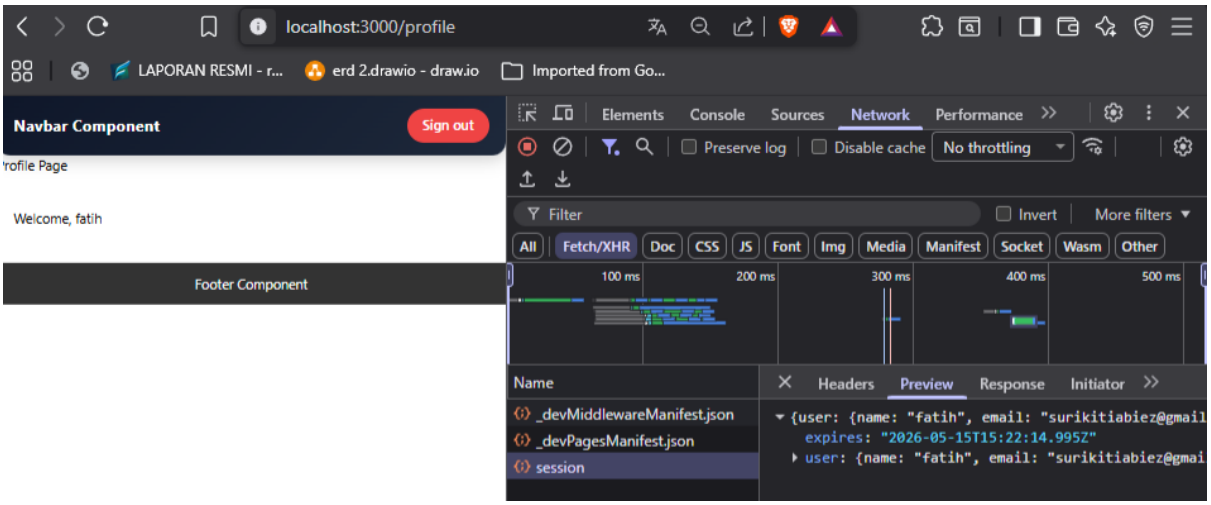
P2 > Praktikum > next-routing > src > middleware.ts > middleware

```
import { getToken } from "next-auth/jwt";
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Route yang memerlukan proteksi autentikasi
const protectedRoutes = ["/product", "/about"];
const profileRoutes = ["/profile"];

export async function middleware(request: NextRequest) {
  // Ambil JWT token dari NextAuth
  const token = await getToken({
    req: request,
    secret: process.env.NEXTAUTH_SECRET,
  });

  const { pathname } = request.nextUrl;

  // Cek apakah route saat ini adalah profile
  const isProfileRoute = profileRoutes.some((route) =>
    pathname.startsWith(route),
  );

  // Cek apakah route saat ini memerlukan proteksi
  const isProtectedRoute = protectedRoutes.some((route) =>
    pathname.startsWith(route),
  );

  // Jika belum login dan mengakses /profile, redirect ke / (home)
  if (!token && isProfileRoute) {
    return NextResponse.redirect(new URL("/", request.url));
  }

  // Jika belum login dan mengakses protected route lainnya, redirect ke /
  if (!token && isProtectedRoute) {
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }

  // Jika sudah login dan mengakses /auth/login, redirect ke /product
  if (token && pathname === "/auth/login") {
    return NextResponse.redirect(new URL("/product", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/product", "/about", "/auth/login", "/profile"],
};
```

6. Dokumentasikan:
   - Screenshot login



- Screenshot session



- Screenshot redirect middleware

```
// Jika belum login dan mengakses /profile, redirect ke / (home)
if (!token && isProfileRoute) {
  return NextResponse.redirect(new URL("/", request.url));
}
```

PERTANYAAN ANALISIS

1. **Mengapa session menggunakan JWT?**
   Session menggunakan JWT (*JSON Web Token*) karena bersifat stateless, artinya server tidak perlu menyimpan data session di database atau memory; semua informasi sudah tersimpan di dalam token yang dikirim ke client. Ini membuat aplikasi lebih scalable, terutama untuk arsitektur modern seperti REST API atau Next.js, serta mempermudah komunikasi antar service tanpa harus cek session ke server terus-menerus.

2. **Apa perbedaan authorize() dan callback jwt()?**
   authorize() digunakan saat proses login untuk memvalidasi kredensial user (misalnya email & password) dan menentukan apakah user boleh masuk atau tidak. Sedangkan callback jwt() digunakan setelah login berhasil untuk menyimpan atau memodifikasi data yang akan dimasukkan ke dalam token JWT (misalnya id user, role, dll). Jadi, authorize() = validasi login, jwt() = pengelolaan isi token.

3. **Mengapa middleware perlu getToken()?**
   Middleware perlu getToken() untuk mengambil dan membaca JWT dari request (biasanya dari cookie atau header). Dengan begitu middleware bisa mengecek apakah user sudah login atau belum sebelum mengakses halaman tertentu, tanpa perlu query ke database lagi—lebih cepat dan efisien.

4. **Apa risiko jika NEXTAUTH_SECRET tidak digunakan?**
   Tanpa NEXTAUTH_SECRET, JWT tidak akan terenkripsi/tertandatangani dengan aman, sehingga token bisa dipalsukan atau dimanipulasi oleh pihak tidak bertanggung jawab. Ini sangat berbahaya karena bisa memungkinkan seseorang mengakses sistem sebagai user lain (bahkan admin).

5. **Apa perbedaan autentikasi dan otorisasi dalam sistem ini?**
   Autentikasi adalah proses memastikan identitas user (contoh: login dengan email & password). Sedangkan otorisasi adalah proses menentukan hak akses user setelah berhasil login (contoh: apakah user boleh mengakses halaman admin atau tidak). Singkatnya: autentikasi = “siapa kamu?”, otorisasi = “apa yang boleh kamu lakukan?”.